Lecture 1 – DS and Algs

25.02.2026

- Moodle quiz after each lecture (optional)

(in general. Not Python, not C++ only, but pseudocode)

Abstract data types → flexibility (don't change the whole thing; 3 → 1)

Data type → ex: array, integer, float…
→ def: set of values together with a set of operations
= INTERFACE

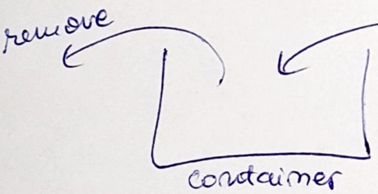
DATA TYPES
- BUILT-IN DT.
- USER-DEFINED DT. (ex: fraction → struct)

+ ABSTRACT
(no testing, implementation) → WHAT, not HOW.

Ex: Data ADT → year
(of user-def. ADT.) → month
→ day } domain

+ difference (d1, d2) } interface ~~HOW~~)

C++ container = collection of data → ≠ types.



→ access elements using ITERATOR.

Containers = Collections
(C++) (Python, …)

CONTAINERS

LIST
- not unique
- have positions
- (∀) el. accessible
- simple elem.

DICTIONARY
- Keys are unique
- no positions (Key ≠ pos.)
- (∀) el. accessible (by Key)
- not simple elem (Key-val e

ADT → I don't know ~~many~~ anything abt implementation, but I know enough info abt domain/ other things to implement it.

Ex: Roman Numerals → key-value pairs → dictionary

? but I need { key → value
value → key

Implementation:

↳ How to STORE data into computer.

DS

STATIC (fixed size) | DYNAMIC (size varies)

Data structures vs. Data types vs. Abstract Data types.

```
if elem in container
    print ("Found")
```

LIST ← → DICTIONARY

≠ complexity

Implement a dictionary yourself

PSEUDOCODE

- // comment
- ←
- read
- execute
- ↑ → pointer

! Indexing is done from 1 in Pseudocode

↓

not OOP

(structures, not classes)

TElem → generic type of a container ADT.

TComp → comparison → 3 outputs → <= ?
→ >
→ =

RAM model: → simple operations : +, -, *, /, =, if, function call

↓

simplified model of how computers work.

How many steps:

$T(n) = 1 + 2*n*(1+n+1)+1 = 2n^2+4n+2$.

↳ not the complexity! (intermediate step) ↳ nr. of steps.

Leading term

n^2

"grows at fast as n^2 does" (still not the complexity)

Complexity: HOW DO THE NR. OF STEPS CHANGE "if I increase n" →? (decrease)

$\boxed{O(\dots) \to \text{upper bound}}$ / $\boxed{\Omega(\dots) \to \text{lower bound}}$ → not useful.

$2n^2 < 4n^2$ → find a trashhold starting from which this inequality holds?

! don't care abt. the ct.

I can use the O notation as long as ... ~~bigger~~ smaller than .. n^2

? leading term → total complexity (overall)

$T(n) = n^2 + 2n + 2$

$T(n) = O(n^2)$ bc. $T(n) \leq c*n^2$ for $c=2$

$T(n) = O(n^3)$ bc. → ~~not useful~~!

$\boxed{\Theta(\dots) \to \text{actual value}}$ → Leading term → $\Theta(n^2)$.

$\Theta(n)$ → n steps.

$O(n)$ → 2n steps / $\frac{1}{2}$ n steps / log....

sum of elems. in array	first occurrence
length of the array counts!	length does not count (matters if it's position) ↳ AC, BC, WC. → Θ → Θ( ), →O( ), →Ω( )

AC formula

$P(I)$ = probability of having I as input

E: $E(I)$ = nr. of operations performed by alg. for input I]

$$\boxed{\sum_{I \in D} P(I) \cdot E(I)}$$

ex: "how many times we perform the loop?"

one input data

domain of the problem (the set of all possible input)

? $\frac{1}{n+1}$ → probability for every input. (the same always?)

in our case

$\cancel{\exists}$ $O(1)$

Recursive Binary Search

good to always have sth. along $T(\ldots)$

$$T(n) = \begin{cases} 1, & n \le 1 \\ T\left(\frac{n}{2}\right)+1, & \text{otherwise} \end{cases}$$

$n = 2^k$ (assume) for easy comp.

$$T(2^k) = T(2^{k-1}) + 1$$
$$T(2^{k-1}) = T(2^{k-2}) + 1$$
$$\vdots$$
$$T(2^1) = T(2^0) + 1$$

(+)

$$T(2^k) = \underbrace{T(2^0)}_{1} + \underbrace{1 + \ldots + 1}_{k \text{ 1's}} = 1 + k$$

why?

$T(n) = n$?!

$n = 2^k \Rightarrow k = \log_2 n \Rightarrow T(n) = 1 + \log_2 n \in \Theta(\log_2 n)$